IncidentDesk

From laptop to a monitored Kubernetes deployment

DevOps Final Capstone · Session 21
github.com/BinaryBhakti/incidentdesk-devops-capstone

1 · Project overview

- **IncidentDesk**: an incident-management SaaS. Open incidents (service, SEV1–SEV4), move them OPEN → INVESTIGATING → RESOLVED, and watch live KPIs
- Responsive React dashboard: KPI cards, filters, incident table, create modal
- FastAPI REST API with 9 endpoints, plus `/health`, `/ready` and `/metrics`
- PostgreSQL schema owned by Alembic migrations
- Goal: show how one commit travels **Git** → **CI** → **security gate** → **registry** → **Terraform** → **Kubernetes** → **monitoring**, and how to troubleshoot it when it breaks

2 · Tech stack

Application

Python 3.12 · FastAPI · SQLAlchemy 2 · Alembic · PostgreSQL 16 · pytest · ruff · React 19 · Vite · nginx

Delivery

Docker · Compose · GitHub Actions · GHCR · Trivy · gitleaks

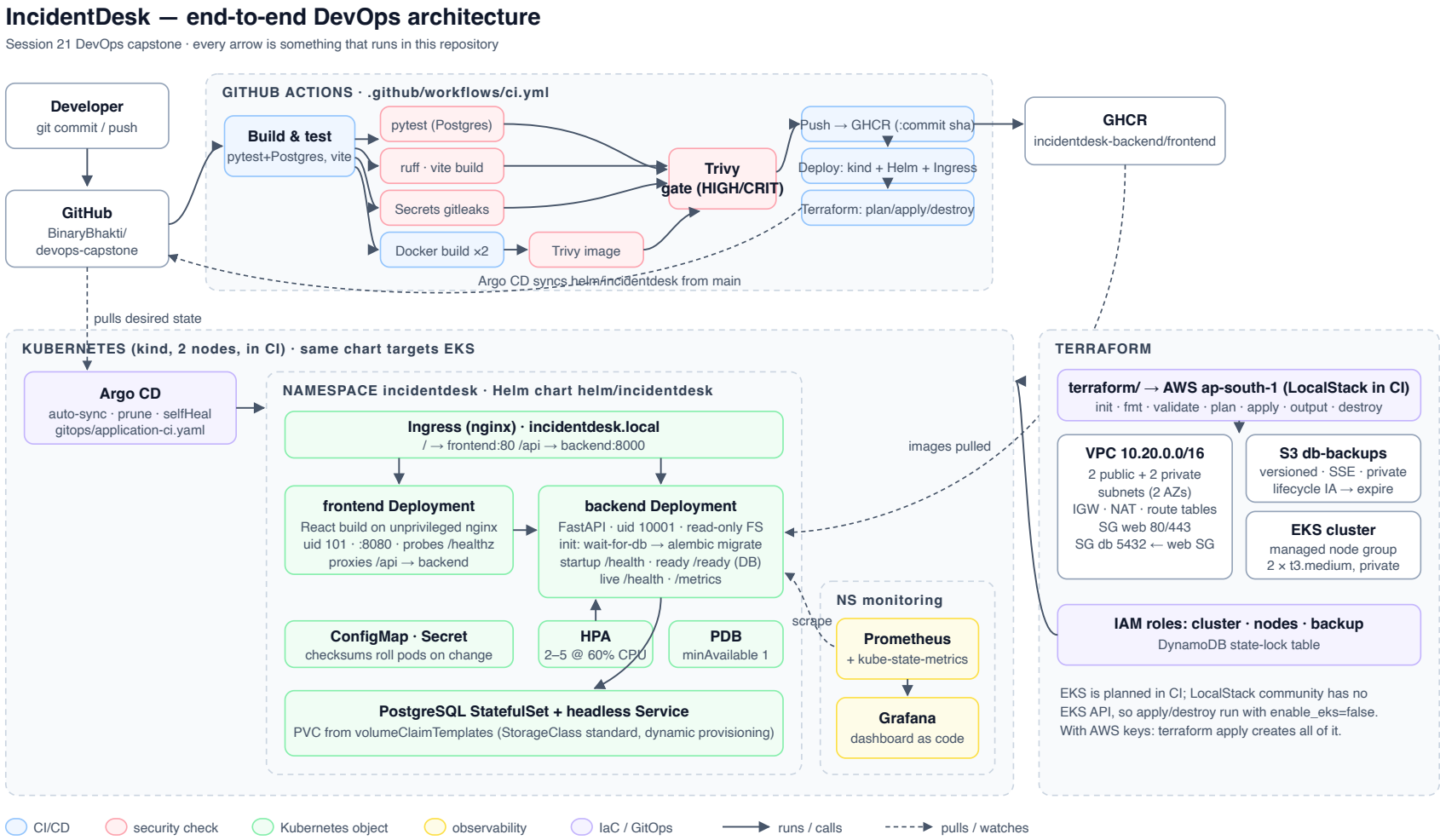
Platform

Terraform · AWS VPC · EKS · LocalStack · Kubernetes (kind) · Helm · NGINX Ingress · HPA + metrics-server · Argo CD

Observability

Prometheus · kube-state-metrics · Grafana · alert rules · dashboard as code

3 · Architecture



Flow: commit → CI tests, scans and builds → gate → image to GHCR → CI deploys to a kind cluster via Helm →
Ingress routes / and /api → HPA scales the backend → Argo CD syncs from Git → Prometheus scrapes /metrics → Grafana shows golden signals.

4 · CI/CD pipeline (GitHub Actions)

```
push to main
  ├─ Test ............ ruff · pytest -v (real Postgres service) · npm ci && vite build
  ├─ Secret scan ..... gitleaks over the full history
  ├─ Build/scan/push . docker build ×2 → Trivy (HIGH/CRITICAL, exit 1) → GHCR :<commit sha>
  ├─ Terraform ....... fmt · init · validate · plan (VPC+EKS) · apply · destroy (LocalStack)
  └─ Deploy .......... kind → Ingress → Prometheus/Grafana → helm upgrade --install
                       → smoke test via Ingress → load test → HPA → PromQL checks
                       → screenshots → Argo CD sync → troubleshooting lab
```

- Each stage is a gate: a failing test or a fixable HIGH CVE means **no image is pushed**
- Image tag = commit SHA, so production traces back to one exact commit

5 · DevSecOps

Layer	Tool	Where
Secret scanning	gitleaks (full Git history)	CI job *Secret scan*
Container scanning	Trivy: OS packages + Python/npm deps	both images, before push
SAST / dependency audit	Semgrep, Bandit, pip-audit configs	`security/`
Runtime hardening	non-root uid, read-only root FS, no default DB password	Dockerfiles, Helm chart

A scanner is one control, not a guarantee. `--ignore-unfixed` gates on what we can fix by upgrading. Accepted risks need a reason and a review date in `.trivyignore`.

6 · Infrastructure as Code: Terraform

- VPC `10.20.0.0/16` in **ap-south-1**
- 2 public + 2 private subnets across 2 AZs, IGW, NAT, route tables
- **EKS cluster** + **managed node group** (2 × t3.medium, private subnets)
- IAM roles, security groups, S3 backups bucket, DynamoDB lock table

```
terraform init
terraform plan   # VPC + EKS
terraform apply
aws eks update-kubeconfig \
  --name incidentdesk-dev-eks
terraform destroy
```

No AWS account: CI runs it on LocalStack. The plan includes EKS; apply/destroy create and remove the network, IAM and storage (EKS is LocalStack Pro-only).

7 · Kubernetes + Helm

- Namespace `incidentdesk`
- Backend + frontend Deployments, **2 replicas** each, rolling updates
- Startup / liveness `/health` · readiness `/ready`
- PostgreSQL StatefulSet + PVC
- ClusterIP Services · Ingress `/` → frontend, `/api` → backend
- HPA 2–5 pods at 60% CPU · PDB

```
kubectl apply -f k8s/namespace.yaml
helm upgrade --install incidentdesk \
  ./helm/incidentdesk -n incidentdesk
kubectl get pods,svc,ingress,hpa \
  -n incidentdesk
helm list -n incidentdesk
```

GitOps: an Argo CD Application syncs the same chart from Git with auto-sync, prune and self-heal.

8 · Observability

- `/metrics`: request count and latency histogram per route, plus `incidents_created_total`
- **Prometheus** scrapes annotated pods and kube-state-metrics; 6 alert rules (backend down, not ready, restarts, error rate, p95 latency, HPA at max)
- **Grafana** dashboard provisioned from code: pods up, req/s, error ratio, p50/p95/p99, CPU and memory per pod, HPA replicas
- Answers: is traffic arriving? which endpoint is slow? are errors rising? is the HPA scaling?

9 · Troubleshooting

Break	Symptom	Investigation	Fix
Image tag that doesn't exist	ImagePullBackOff	describe pod, events	set image to the CI SHA
Service selector typo	no endpoints → 502	get endpoints, --show-labels	match labels
Wrong readiness path	0/1 Ready	describe, probe events	/ready
HPA without requests	<unknown> targets	describe hpa	set requests
Hand edit in cluster	drift	Argo CD diff	self-heal

10 · Key learnings

- Gates belong **before** the registry: tests and the scanner decide what is allowed to exist
- Liveness ≠ readiness: `/ready` checks the database, so traffic only reaches pods that can serve
- SHA tags beat `latest`: rollback means picking a commit
- A Service is only labels. Most "network" bugs are selector bugs
- HPA needs requests and a metrics provider; Ingress needs a controller
- Declare everything (Terraform, Helm, dashboards) so a rebuild looks identical
- Be honest about limits: LocalStack and kind stand in for AWS and EKS here